```
1     ;/*
2     ;       derived from: FreeBSD @(#)syscalls.master       8.2 (Berkeley) 1/13/94
3     ;
4     ; System call name/number master file.
5     ; This is file processed by .../xnu/bsd/kern/makesyscalls.sh and creates:
6     ;       .../xnu/bsd/kern/init_sysent.c
7     ;       .../xnu/bsd/kern/syscalls.c
8     ;       .../xnu/bsd/sys/syscall.h
9     ;       .../xnu/bsd/sys/sysproto.h
10    ;       .../xnu/bsd/security/audit_syscalls.c
11
12    ; Columns -> | Number Audit Files | { Name and Args } | { Comments }
13    ;       Number:         system call number, must be in order
14    ;       Audit:          the audit event associated with the system call
15    ;                       A value of AUE_NULL means no auditing, but it also means that
16    ;                       there is no audit event for the call at this time. For the
17    ;                       case where the event exists, but we don't want auditing, the
18    ;                       event should be #defined to AUE_NULL in audit_kevents.h.
19    ;       Files:          with files to generate - "ALL" or any combo of:
20    ;                                       "T" for syscall table (in init_sysent.c)
21    ;                                       "N" for syscall names (in syscalls.c)
22    ;                                       "H" for syscall headers (in syscall.h)
23    ;                                       "P" for syscall prototypes (in sysproto.h)
24    ;       Name and Args:  function prototype, optionally followed by
25    ;                       NO_SYSCALL_STUB (which mean no system call stub will
26    ;                       be generated in libSystem) and ending with a semicolon.
27    ;                       (Note: functions prefixed by double-underbar are
28    ;                       automatically given the NO_SYSCALL_STUB attribute.)
29    ;       Comments:       additional comments about the sys call copied to output files
30
```

Code Blame Raw

```
36      #include <sys/appleapiopts.h>
37      #include <sys/param.h>
38      #include <sys/systm.h>
39      #include <sys/types.h>
40      #include <sys/sysent.h>
41      #include <sys/sysproto.h>
42      #include <nfs/nfs_conf.h>
43
44      0       AUE_NULL        ALL     { int nosys(void); }   { indirect syscall }
45      1       AUE_EXIT        ALL     { void exit(int rval) NO_SYSCALL_STUB; }
46      2       AUE_FORK        ALL     { int fork(void) NO_SYSCALL_STUB; }
47      3       AUE_NULL        ALL     { user_ssize_t read(int fd, user_addr_t cbuf, user_size_t
48      4       AUE_NULL        ALL     { user_ssize_t write(int fd, user_addr_t cbuf, user_size_t
49      5       AUE_OPEN_RWTC   ALL     { int open(user_addr_t path, int flags, int mode) NO_SYSCA
50      6       AUE_CLOSE       ALL     { int sys_close(int fd); }
51      7       AUE_WAIT4       ALL     { int wait4(int pid, user_addr_t status, int options, user
52      8       AUE_NULL        ALL     { int enosys(void); }   { old creat }
53      9       AUE_LINK        ALL     { int link(user_addr_t path, user_addr_t link); }
54      10      AUE_UNLINK      ALL     { int unlink(user_addr_t path) NO_SYSCALL_STUB; }
55      11      AUE_NULL        ALL     { int enosys(void); }   { old execv }
56      12      AUE_CHDIR       ALL     { int chdir(user_addr_t path); }
57      13      AUE_FCHDIR      ALL     { int fchdir(int fd); }
58      14      AUE_MKNOD       ALL     { int mknod(user_addr_t path, int mode, int dev); }
59      15      AUE_CHMOD       ALL     { int chmod(user_addr_t path, int mode) NO_SYSCALL_STUB; }
60      16      AUE_CHOWN       ALL     { int chown(user_addr_t path, int uid, int gid); }
61      17      AUE_NULL        ALL     { int enosys(void); }   { old break }
62      18      AUE_GETFSSTAT   ALL     { int getfsstat(user_addr_t buf, int bufsize, int flags);
63      19      AUE_NULL        ALL     { int enosys(void); }   { old lseek }
64      20      AUE_GETPID      ALL     { int getpid(void); }
65      21      AUE_NULL        ALL     { int enosys(void); }   { old mount }
66      22      AUE_NULL        ALL     { int enosys(void); }   { old umount }
67      23      AUE_SETUID      ALL     { int setuid(uid_t uid); }
68      24      AUE_GETUID      ALL     { int getuid(void); }
69      25      AUE_GETEUID     ALL     { int geteuid(void); }
70      26      AUE_PTRACE      ALL     { int ptrace(int req, pid_t pid, caddr_t addr, int data);
71      #if SOCKETS
72      27      AUE_RECVMSG     ALL     { int recvmsg(int s, struct msghdr *msg, int flags) NO_SYS
73      28      AUE_SENDMSG     ALL     { int sendmsg(int s, caddr_t msg, int flags) NO_SYSCALL_ST
74      29      AUE_RECVFROM    ALL     { int recvfrom(int s, void *buf, size_t len, int flags, st
75      30      AUE_ACCEPT      ALL     { int accept(int s, caddr_t name, socklen_t     *anamelen)
76      31      AUE_GETPEERNAME ALL     { int getpeername(int fdes, caddr_t asa, socklen_t *alen)
77      32      AUE_GETSOCKNAME ALL     { int getsockname(int fdes, caddr_t asa, socklen_t *alen)
78      #else
79      27      AUE_NULL        ALL     { int nosys(void); }
80      28      AUE_NULL        ALL     { int nosys(void); }
81      29      AUE_NULL        ALL     { int nosys(void); }
83      31      AUE_NULL        ALL     { int nosys(void); }
84      32      AUE_NULL        ALL     { int nosys(void); }
85      #endif /* SOCKETS */
```

```
#endif /* SOCKETS */
86	33	AUE_ACCESS	ALL	{ int access(user_addr_t path, int flags); }
87	34	AUE_CHFLAGS	ALL	{ int chflags(char *path, int flags); }
88	35	AUE_FCHFLAGS	ALL	{ int fchflags(int fd, int flags); }
89	36	AUE_SYNC	ALL	{ int sync(void); }
90	37	AUE_KILL	ALL	{ int kill(int pid, int signum, int posix) NO_SYSCALL_STUB
91	38	AUE_NULL	ALL	{ int nosys(void); }   { old stat  }
92	39	AUE_GETPPID	ALL	{ int getppid(void); }
93	40	AUE_NULL	ALL	{ int nosys(void); }   { old lstat }
94	41	AUE_DUP	ALL	{ int sys_dup(u_int fd); }
95	42	AUE_PIPE	ALL	{ int pipe(void); }
96	43	AUE_GETEGID	ALL	{ int getegid(void); }
97	44	AUE_NULL	ALL	{ int nosys(void); } { old profil }
98	45	AUE_NULL	ALL	{ int nosys(void); } { old ktrace }
99	46	AUE_SIGACTION	ALL	{ int sigaction(int signum, struct __sigaction *nsa, struc
100	47	AUE_GETGID	ALL	{ int getgid(void); }
101	48	AUE_SIGPROCMASK	ALL	{ int sigprocmask(int how, user_addr_t mask, user_addr_t o
102	49	AUE_GETLOGIN	ALL	{ int getlogin(char *namebuf, u_int namelen) NO_SYSCALL_ST
103	50	AUE_SETLOGIN	ALL	{ int setlogin(char *namebuf) NO_SYSCALL_STUB; }
104	51	AUE_ACCT	ALL	{ int acct(char *path); }
105	52	AUE_SIGPENDING	ALL	{ int sigpending(struct sigvec *osv); }
106	53	AUE_SIGALTSTACK	ALL	{ int sigaltstack(struct sigaltstack *nss, struct sigaltst
107	54	AUE_IOCTL	ALL	{ int ioctl(int fd, u_long com, caddr_t data) NO_SYSCALL_S
108	55	AUE_REBOOT	ALL	{ int reboot(int opt, char *msg) NO_SYSCALL_STUB; }
109	56	AUE_REVOKE	ALL	{ int revoke(char *path); }
110	57	AUE_SYMLINK	ALL	{ int symlink(char *path, char *link); }
111	58	AUE_READLINK	ALL	{ int readlink(char *path, char *buf, int count); }
112	59	AUE_EXECVE	ALL	{ int execve(char *fname, char **argp, char **envp); }
113	60	AUE_UMASK	ALL	{ int umask(int newmask); }
114	61	AUE_CHROOT	ALL	{ int chroot(user_addr_t path); }
115	62	AUE_NULL	ALL	{ int nosys(void); }   { old fstat }
116	63	AUE_NULL	ALL	{ int nosys(void); }   { used internally and reserved }
117	64	AUE_NULL	ALL	{ int nosys(void); }   { old getpagesize }
118	65	AUE_MSYNC	ALL	{ int msync(caddr_t addr, size_t len, int flags) NO_SYSCAL
119	66	AUE_VFORK	ALL	{ int vfork(void); }
120	67	AUE_NULL	ALL	{ int nosys(void); }   { old vread }
121	68	AUE_NULL	ALL	{ int nosys(void); }   { old vwrite }
122	69	AUE_NULL	ALL	{ int nosys(void); }   { old sbrk }
123	70	AUE_NULL	ALL	{ int nosys(void); }   { old sstk }
124	71	AUE_NULL	ALL	{ int nosys(void); }   { old mmap }
125	72	AUE_NULL	ALL	{ int nosys(void); }   { old vadvise }
126	73	AUE_MUNMAP	ALL	{ int munmap(caddr_t addr, size_t len) NO_SYSCALL_STUB; }
127	74	AUE_MPROTECT	ALL	{ int mprotect(caddr_t addr, size_t len, int prot) NO_SYSC
128	75	AUE_MADVISE	ALL	{ int madvise(caddr_t addr, size_t len, int behav); }
129	76	AUE_NULL	ALL	{ int nosys(void); }   { old vhangup }
130	77	AUE_NULL	ALL	{ int nosys(void); }   { old vlimit }
131	78	AUE_MINCORE	ALL	{ int mincore(user_addr_t addr, user_size_t len, user_addr
132	79	AUE_GETGROUPS	ALL	{ int getgroups(u_int gidsetsize, gid_t *gidset); }
133	80	AUE_SETGROUPS	ALL	{ int setgroups(u_int gidsetsize, gid_t *gidset); }
134	81	AUE_GETPGRP	ALL	{ int getpgrp(void); }
135	82	AUE_SETPGRP	ALL	{ int setpgid(int pid, int pgid); }
136	83	AUE_SETITIMER	ALL	{ int setitimer(u_int which, struct itimerval *itv, struct
137	84	AUE_NULL	ALL	{ int nosys(void); }   { old wait }
```

```
138     85      AUE_SWAPON      ALL     { int swapon(void); }
139     86      AUE_GETITIMER   ALL     { int getitimer(u_int which, struct itimerval *itv); }
140     87      AUE_NULL        ALL     { int nosys(void); }   { old gethostname }
141     88      AUE_NULL        ALL     { int nosys(void); }   { old sethostname }
142     89      AUE_GETDTABLESIZE       ALL     { int sys_getdtablesize(void); }
143     90      AUE_DUP2        ALL     { int sys_dup2(u_int from, u_int to); }
144     91      AUE_NULL        ALL     { int nosys(void); }   { old getdopt }
145     92      AUE_FCNTL       ALL     { int sys_fcntl(int fd, int cmd, long arg) NO_SYSCALL_STUB
146     93      AUE_SELECT      ALL     { int select(int nd, u_int32_t *in, u_int32_t *ou, u_int32
147     94      AUE_NULL        ALL     { int nosys(void); }   { old setdopt }
148     95      AUE_FSYNC       ALL     { int fsync(int fd); }
149     96      AUE_SETPRIORITY ALL     { int setpriority(int which, id_t who, int prio) NO_SYSCAL
150     #if SOCKETS
151     97      AUE_SOCKET      ALL     { int socket(int domain, int type, int protocol); }
152     98      AUE_CONNECT     ALL     { int connect(int s, caddr_t name, socklen_t namelen) NO_S
153     #else
154     97      AUE_NULL        ALL     { int nosys(void); }
155     98      AUE_NULL        ALL     { int nosys(void); }
156     #endif /* SOCKETS */
157     99      AUE_NULL        ALL     { int nosys(void); }   { old accept }
158     100     AUE_GETPRIORITY ALL     { int getpriority(int which, id_t who); }
159     101     AUE_NULL        ALL     { int nosys(void); }   { old send }
160     102     AUE_NULL        ALL     { int nosys(void); }   { old recv }
161     103     AUE_NULL        ALL     { int nosys(void); }   { old sigreturn }
162     #if SOCKETS
163     104     AUE_BIND        ALL     { int bind(int s, caddr_t name, socklen_t namelen) NO_SYSC
164     105     AUE_SETSOCKOPT  ALL     { int setsockopt(int s, int level, int name, caddr_t val,
165     106     AUE_LISTEN      ALL     { int listen(int s, int backlog) NO_SYSCALL_STUB; }
166     #else
167     104     AUE_NULL        ALL     { int nosys(void); }
168     105     AUE_NULL        ALL     { int nosys(void); }
169     106     AUE_NULL        ALL     { int nosys(void); }
170     #endif /* SOCKETS */
171     107     AUE_NULL        ALL     { int nosys(void); }   { old vtimes }
172     108     AUE_NULL        ALL     { int nosys(void); }   { old sigvec }
173     109     AUE_NULL        ALL     { int nosys(void); }   { old sigblock }
174     110     AUE_NULL        ALL     { int nosys(void); }   { old sigsetmask }
175     111     AUE_NULL        ALL     { int sigsuspend(sigset_t mask) NO_SYSCALL_STUB; }
176     112     AUE_NULL        ALL     { int nosys(void); }   { old sigstack }
177     #if SOCKETS
178     113     AUE_NULL        ALL     { int nosys(void); }   { old recvmsg }
179     114     AUE_NULL        ALL     { int nosys(void); }   { old sendmsg }
180     #else
181     113     AUE_NULL        ALL     { int nosys(void); }
182     114     AUE_NULL        ALL     { int nosys(void); }
183     #endif /* SOCKETS */
184     115     AUE_NULL        ALL     { int nosys(void); }   { old vtrace }
185     116     AUE_GETTIMEOFDAY        ALL     { int gettimeofday(struct timeval *tp, struct time
186     117     AUE_GETRUSAGE   ALL     { int getrusage(int who, struct rusage *rusage); }
187     #if SOCKETS
188     118     AUE_GETSOCKOPT  ALL     { int getsockopt(int s, int level, int name, caddr_t val,
189     #else
```

```
190     118     AUE_NULL        ALL     { int nosys(void); }
191     #endif /* SOCKETS */
192     119     AUE_NULL        ALL     { int nosys(void); }   { old resuba }
193     120     AUE_READV       ALL     { user_ssize_t readv(int fd, struct iovec *iovp, u_int iov
194     121     AUE_WRITEV      ALL     { user_ssize_t writev(int fd, struct iovec *iovp, u_int io
195     122     AUE_SETTIMEOFDAY        ALL     { int settimeofday(struct timeval *tv, struct time
196     123     AUE_FCHOWN      ALL     { int fchown(int fd, int uid, int gid); }
197     124     AUE_FCHMOD      ALL     { int fchmod(int fd, int mode) NO_SYSCALL_STUB; }
198     125     AUE_NULL        ALL     { int nosys(void); }   { old recvfrom }
199     126     AUE_SETREUID    ALL     { int setreuid(uid_t ruid, uid_t euid) NO_SYSCALL_STUB; }
200     127     AUE_SETREGID    ALL     { int setregid(gid_t rgid, gid_t egid) NO_SYSCALL_STUB; }
201     128     AUE_RENAME      ALL     { int rename(char *from, char *to) NO_SYSCALL_STUB; }
202     129     AUE_NULL        ALL     { int nosys(void); }   { old truncate }
203     130     AUE_NULL        ALL     { int nosys(void); }   { old ftruncate }
204     131     AUE_FLOCK       ALL     { int sys_flock(int fd, int how); }
205     132     AUE_MKFIFO      ALL     { int mkfifo(user_addr_t path, int mode); }
206     #if SOCKETS
207     133     AUE_SENDTO      ALL     { int sendto(int s, caddr_t buf, size_t len, int flags, ca
208     134     AUE_SHUTDOWN    ALL     { int shutdown(int s, int how); }
209     135     AUE_SOCKETPAIR  ALL     { int socketpair(int domain, int type, int protocol, int *
210     #else
211     133     AUE_NULL        ALL     { int nosys(void); }
212     134     AUE_NULL        ALL     { int nosys(void); }
213     135     AUE_NULL        ALL     { int nosys(void); }
214     #endif /* SOCKETS */
215     136     AUE_MKDIR       ALL     { int mkdir(user_addr_t path, int mode); }
216     137     AUE_RMDIR       ALL     { int rmdir(char *path) NO_SYSCALL_STUB; }
217     138     AUE_UTIMES      ALL     { int utimes(char *path, struct timeval *tptr); }
218     139     AUE_FUTIMES     ALL     { int futimes(int fd, struct timeval *tptr); }
219     140     AUE_ADJTIME     ALL     { int adjtime(struct timeval *delta, struct timeval *oldde
220     141     AUE_NULL        ALL     { int nosys(void); }   { old getpeername }
221     142     AUE_SYSCTL      ALL     { int gethostuuid(unsigned char *uuid_buf, const struct ti
222     143     AUE_NULL        ALL     { int nosys(void); }   { old sethostid  }
223     144     AUE_NULL        ALL     { int nosys(void); }   { old getrlimit }
224     145     AUE_NULL        ALL     { int nosys(void); }   { old setrlimit }
225     146     AUE_NULL        ALL     { int nosys(void); }   { old killpg }
226     147     AUE_SETSID      ALL     { int setsid(void); }
227     148     AUE_NULL        ALL     { int nosys(void); }   { old setquota }
228     149     AUE_NULL        ALL     { int nosys(void); }   { old qquota }
229     150     AUE_NULL        ALL     { int nosys(void); }   { old getsockname }
230     151     AUE_GETPGID     ALL     { int getpgid(pid_t pid); }
231     152     AUE_SETPRIVEXEC ALL     { int setprivexec(int flag); }
232     153     AUE_PREAD       ALL     { user_ssize_t pread(int fd, user_addr_t buf, user_size_t
233     154     AUE_PWRITE      ALL     { user_ssize_t pwrite(int fd, user_addr_t buf, user_size_t
234
235     #if NFSSERVER   /* XXX */
236     155     AUE_NFS_SVC     ALL     { int nfssvc(int flag, caddr_t argp); }
237     #else
238     155     AUE_NULL        ALL     { int nosys(void); }
239     #endif
240
241     156     AUE_NULL        ALL     { int nosys(void); }   { old getdirentries }
242     157     AUE_STATFS      ALL     { int statfs(char *path, struct statfs *buf); }
```

```
243     158     AUE_FSTATFS     ALL     { int fstatfs(int fd, struct statfs *buf); }
244     159     AUE_UNMOUNT     ALL     { int unmount(user_addr_t path, int flags); }
245     160     AUE_NULL        ALL     { int nosys(void); }   { old async_daemon }
246
247     #if NFSSERVER   /* XXX */
248     161     AUE_NFS_GETFH   ALL     { int getfh(char *fname, fhandle_t *fhp); }
249     #else
250     161     AUE_NULL        ALL     { int nosys(void); }
251     #endif
252
253     162     AUE_NULL        ALL     { int nosys(void); }   { old getdomainname }
254     163     AUE_NULL        ALL     { int nosys(void); }   { old setdomainname }
255     164     AUE_NULL        ALL     { int nosys(void); }
256     165     AUE_QUOTACTL    ALL     { int quotactl(const char *path, int cmd, int uid, caddr_t
257     166     AUE_NULL        ALL     { int nosys(void); }   { old exportfs }
258     167     AUE_MOUNT       ALL     { int mount(char *type, char *path, int flags, caddr_t dat
259     168     AUE_NULL        ALL     { int nosys(void); }   { old ustat }
260     169     AUE_CSOPS       ALL     { int csops(pid_t pid, uint32_t ops, user_addr_t useraddr,
261     170     AUE_CSOPS       ALL     { int csops_audittoken(pid_t pid, uint32_t ops, user_addr_
262     171     AUE_NULL        ALL     { int nosys(void); }   { old wait3 }
263     172     AUE_NULL        ALL     { int nosys(void); }   { old rpause     }
264     173     AUE_WAITID      ALL     { int waitid(idtype_t idtype, id_t id, siginfo_t *infop, i
265     174     AUE_NULL        ALL     { int nosys(void); }   { old getdents }
266     175     AUE_NULL        ALL     { int nosys(void); }   { old gc_control }
267     176     AUE_NULL        ALL     { int nosys(void); }   { old add_profil }
268     177     AUE_NULL        ALL     { int kdebug_typefilter(void** addr, size_t* size) NO_SYSC
269     178     AUE_NULL        ALL     { uint64_t kdebug_trace_string(uint32_t debugid, uint64_t
270     179     AUE_NULL        ALL     { int kdebug_trace64(uint32_t code, uint64_t arg1, uint64_
271     180     AUE_NULL        ALL     { int kdebug_trace(uint32_t code, u_long arg1, u_long arg2
272     181     AUE_SETGID      ALL     { int setgid(gid_t gid); }
273     182     AUE_SETEGID     ALL     { int setegid(gid_t egid); }
274     183     AUE_SETEUID     ALL     { int seteuid(uid_t euid); }
275     184     AUE_SIGRETURN   ALL     { int sigreturn(struct ucontext *uctx, int infostyle, user
276     185     AUE_NULL        ALL     { int enosys(void); } { old chud }
277     186     AUE_NULL        ALL     { int thread_selfcounts(int type, user_addr_t buf, user_si
278     187     AUE_FDATASYNC   ALL     { int fdatasync(int fd); }
279     188     AUE_STAT        ALL     { int stat(user_addr_t path, user_addr_t ub); }
280     189     AUE_FSTAT       ALL     { int sys_fstat(int fd, user_addr_t ub); }
281     190     AUE_LSTAT       ALL     { int lstat(user_addr_t path, user_addr_t ub); }
282     191     AUE_PATHCONF    ALL     { int pathconf(char *path, int name); }
283     192     AUE_FPATHCONF   ALL     { int sys_fpathconf(int fd, int name); }
284     193     AUE_NULL        ALL     { int nosys(void); }    { old getfsstat }
285     194     AUE_GETRLIMIT   ALL     { int getrlimit(u_int which, struct rlimit *rlp) NO_SYSCAL
286     195     AUE_SETRLIMIT   ALL     { int setrlimit(u_int which, struct rlimit *rlp) NO_SYSCAL
287     196     AUE_GETDIRENTRIES       ALL     { int getdirentries(int fd, char *buf, u_int count
288     197     AUE_MMAP        ALL     { user_addr_t mmap(caddr_t addr, size_t len, int prot, int
289     198     AUE_NULL        ALL     { int nosys(void); }    { old __syscall }
290     199     AUE_LSEEK       ALL     { off_t lseek(int fd, off_t offset, int whence); }
291     200     AUE_TRUNCATE    ALL     { int truncate(char *path, off_t length); }
292     201     AUE_FTRUNCATE   ALL     { int ftruncate(int fd, off_t length); }
293     202     AUE_SYSCTL      ALL     { int sysctl(int *name, u_int namelen, void *old, size_t *
294     203     AUE_MLOCK       ALL     { int mlock(caddr_t addr, size_t len); }
```

```
295     204     AUE_MUNLOCK     ALL     { int munlock(caddr_t addr, size_t len); }
296     205     AUE_UNDELETE    ALL     { int undelete(user_addr_t path); }
297
298     206     AUE_NULL        ALL     { int nosys(void); } { old ATsocket }
299     207     AUE_NULL        ALL     { int nosys(void); } { old ATgetmsg }
300     208     AUE_NULL        ALL     { int nosys(void); } { old ATputmsg }
301     209     AUE_NULL        ALL     { int nosys(void); } { old ATsndreq }
302     210     AUE_NULL        ALL     { int nosys(void); } { old ATsndrsp }
303     211     AUE_NULL        ALL     { int nosys(void); } { old ATgetreq }
304     212     AUE_NULL        ALL     { int nosys(void); } { old ATgetrsp }
305     213     AUE_NULL        ALL     { int nosys(void); }    { Reserved for AppleTalk }
306
307     214     AUE_NULL        ALL     { int nosys(void); }
308     215     AUE_NULL        ALL     { int nosys(void); }
309
310     ; System Calls 216 - 230 are reserved for calls to support HFS/HFS Plus
311     ; file system semantics. Currently, we only use 215-227.  The rest is
312     ; for future expansion in anticipation of new MacOS APIs for HFS Plus.
313     ; These calls are not conditionalized because while they are specific
314     ; to HFS semantics, they are not specific to the HFS filesystem.
315     ; We expect all filesystems to recognize the call and report that it is
316     ; not supported or to actually implement it.
317
318     ; 216-> 219 used to be mkcomplex and {f,l}statv variants. They are gone now.
319     216     AUE_NULL        ALL     { int open_dprotected_np(user_addr_t path, int flags, int
320     217     AUE_FSGETPATH_EXTENDED  ALL     { user_ssize_t fsgetpath_ext(user_addr_t buf, size
321     218     AUE_NULL        ALL     { int nosys(void); }    { old lstatv }
322     219     AUE_NULL        ALL     { int nosys(void); }    { old fstatv }
323     220     AUE_GETATTRLIST ALL     { int getattrlist(const char *path, struct attrlist *alist
324     221     AUE_SETATTRLIST ALL     { int setattrlist(const char *path, struct attrlist *alist
325     222     AUE_GETDIRENTRIESATTR   ALL     { int getdirentriesattr(int fd, struct attrlist *a
326     223     AUE_EXCHANGEDATA        ALL     { int exchangedata(const char *path1, const char *
327     224     AUE_NULL                ALL { int nosys(void); } { old checkuseraccess or fsgetpat
328     225     AUE_SEARCHFS    ALL     { int searchfs(const char *path, struct fssearchblock *sea
329     226     AUE_DELETE      ALL     { int delete(user_addr_t path) NO_SYSCALL_STUB; }       {
330     227     AUE_COPYFILE    ALL     { int copyfile(char *from, char *to, int mode, int flags)
331     228     AUE_FGETATTRLIST        ALL     { int fgetattrlist(int fd, struct attrlist *alist,
332     229     AUE_FSETATTRLIST        ALL     { int fsetattrlist(int fd, struct attrlist *alist,
333     230     AUE_POLL        ALL     { int poll(struct pollfd *fds, u_int nfds, int timeout); }
334     231     AUE_NULL        ALL     { int nosys(void); } { old watchevent }
335     232     AUE_NULL        ALL     { int nosys(void); } { old waitevent }
336     233     AUE_NULL        ALL     { int nosys(void); } { old modwatch }
337     234     AUE_GETXATTR    ALL     { user_ssize_t getxattr(user_addr_t path, user_addr_t attr
338     235     AUE_FGETXATTR   ALL     { user_ssize_t fgetxattr(int fd, user_addr_t attrname, use
339     236     AUE_SETXATTR    ALL     { int setxattr(user_addr_t path, user_addr_t attrname, use
340     237     AUE_FSETXATTR   ALL     { int fsetxattr(int fd, user_addr_t attrname, user_addr_t
341     238     AUE_REMOVEXATTR ALL     { int removexattr(user_addr_t path, user_addr_t attrname,
342     239     AUE_FREMOVEXATTR        ALL     { int fremovexattr(int fd, user_addr_t attrname, i
343     240     AUE_LISTXATTR   ALL     { user_ssize_t listxattr(user_addr_t path, user_addr_t nam
344     241     AUE_FLISTXATTR  ALL     { user_ssize_t flistxattr(int fd, user_addr_t namebuf, siz
345     242     AUE_FSCTL       ALL     { int fsctl(const char *path, u_long cmd, caddr_t data, u_
346     243     AUE_INITGROUPS  ALL     { int initgroups(u_int gidsetsize, gid_t *gidset, int gmui
347     244     AUE_POSIX_SPAWN ALL     { int posix_spawn(pid_t *pid, const char *path, const stru
```

```
348     245     AUE_FFSCTL      ALL     { int ffsctl(int fd, u_long cmd, caddr_t data, u_int optio
349     246     AUE_NULL        ALL     { int nosys(void); }
350
351     #if NFSCLIENT   /* XXX */
352     247     AUE_NULL        ALL     { int nfsclnt(int flag, caddr_t argp); }
353     #else
354     247     AUE_NULL        ALL     { int nosys(void); }
355     #endif
356     #if NFSSERVER   /* XXX */
357     248     AUE_FHOPEN      ALL     { int fhopen(const struct fhandle *u_fhp, int flags); }
358     #else
359     248     AUE_NULL        ALL     { int nosys(void); }
360     #endif
361
362     249     AUE_NULL        ALL     { int nosys(void); }
363     250     AUE_MINHERIT    ALL     { int minherit(void *addr, size_t len, int inherit); }
364     #if SYSV_SEM
365     251     AUE_SEMSYS      ALL     { int semsys(u_int which, int a2, int a3, int a4, int a5)
366     #else
367     251     AUE_NULL        ALL     { int nosys(void); }
368     #endif
369     #if SYSV_MSG
370     252     AUE_MSGSYS      ALL     { int msgsys(u_int which, int a2, int a3, int a4, int a5)
371     #else
372     252     AUE_NULL        ALL     { int nosys(void); }
373     #endif
374     #if SYSV_SHM
375     253     AUE_SHMSYS      ALL     { int shmsys(u_int which, int a2, int a3, int a4) NO_SYSCA
376     #else
377     253     AUE_NULL        ALL     { int nosys(void); }
378     #endif
379     #if SYSV_SEM
380     254     AUE_SEMCTL      ALL     { int semctl(int semid, int semnum, int cmd, semun_t arg)
381     255     AUE_SEMGET      ALL     { int semget(key_t key, int     nsems, int semflg); }
382     256     AUE_SEMOP       ALL     { int semop(int semid, struct sembuf *sops, int nsops); }
383     257     AUE_NULL        ALL     { int nosys(void); }    { old semconfig }
384     #else
385     254     AUE_NULL        ALL     { int nosys(void); }
386     255     AUE_NULL        ALL     { int nosys(void); }
387     256     AUE_NULL        ALL     { int nosys(void); }
388     257     AUE_NULL        ALL     { int nosys(void); }
389     #endif
390     #if SYSV_MSG
391     258     AUE_MSGCTL      ALL     { int msgctl(int msqid, int cmd, struct msqid_ds *buf) NO_
392     259     AUE_MSGGET      ALL     { int msgget(key_t key, int msgflg); }
393     260     AUE_MSGSND      ALL     { int msgsnd(int msqid, void *msgp, size_t msgsz, int msgf
394     261     AUE_MSGRCV      ALL     { user_ssize_t msgrcv(int msqid, void *msgp, size_t msgsz,
395     #else
396     258     AUE_NULL        ALL     { int nosys(void); }
397     259     AUE_NULL        ALL     { int nosys(void); }
398     260     AUE_NULL        ALL     { int nosys(void); }
399     261     AUE_NULL        ALL     { int nosys(void); }
```

```
400     #endif
401     #if SYSV_SHM
402     262     AUE_SHMAT       ALL     { user_addr_t shmat(int shmid, void *shmaddr, int shmflg);
403     263     AUE_SHMCTL      ALL     { int shmctl(int shmid, int cmd, struct shmid_ds *buf) NO_
404     264     AUE_SHMDT       ALL     { int shmdt(void *shmaddr); }
405     265     AUE_SHMGET      ALL     { int shmget(key_t key, size_t size, int shmflg); }
406     #else
407     262     AUE_NULL        ALL     { int nosys(void); }
408     263     AUE_NULL        ALL     { int nosys(void); }
409     264     AUE_NULL        ALL     { int nosys(void); }
410     265     AUE_NULL        ALL     { int nosys(void); }
411     #endif
412     266     AUE_SHMOPEN     ALL     { int shm_open(const char *name, int oflag, int mode) NO_S
413     267     AUE_SHMUNLINK   ALL     { int shm_unlink(const char *name); }
414     268     AUE_SEMOPEN     ALL     { user_addr_t sem_open(const char *name, int oflag, int mo
415     269     AUE_SEMCLOSE    ALL     { int sem_close(sem_t *sem); }
416     270     AUE_SEMUNLINK   ALL     { int sem_unlink(const char *name); }
417     271     AUE_SEMWAIT     ALL     { int sem_wait(sem_t *sem); }
418     272     AUE_SEMTRYWAIT  ALL     { int sem_trywait(sem_t *sem); }
419     273     AUE_SEMPOST     ALL     { int sem_post(sem_t *sem); }
420     274     AUE_SYSCTL      ALL     { int sys_sysctlbyname(const char *name, size_t namelen, v
421     275     AUE_NULL        ALL     { int enosys(void); } { old sem_init }
422     276     AUE_NULL        ALL     { int enosys(void); } { old sem_destroy }
423     277     AUE_OPEN_EXTENDED_RWTC  ALL     { int open_extended(user_addr_t path, int flags, u
424     278     AUE_UMASK_EXTENDED      ALL     { int umask_extended(int newmask, user_addr_t xsec
425     279     AUE_STAT_EXTENDED       ALL     { int stat_extended(user_addr_t path, user_addr_t
426     280     AUE_LSTAT_EXTENDED      ALL     { int lstat_extended(user_addr_t path, user_addr_t
427     281     AUE_FSTAT_EXTENDED      ALL     { int sys_fstat_extended(int fd, user_addr_t ub, u
428     282     AUE_CHMOD_EXTENDED      ALL     { int chmod_extended(user_addr_t path, uid_t uid,
429     283     AUE_FCHMOD_EXTENDED     ALL     { int fchmod_extended(int fd, uid_t uid, gid_t gid
430     284     AUE_ACCESS_EXTENDED     ALL     { int access_extended(user_addr_t entries, size_t
431     285     AUE_SETTID      ALL     { int settid(uid_t uid, gid_t gid) NO_SYSCALL_STUB; }
432     286     AUE_GETTID      ALL     { int gettid(uid_t *uidp, gid_t *gidp) NO_SYSCALL_STUB; }
433     287     AUE_SETSGROUPS  ALL     { int setsgroups(int setlen, user_addr_t guidset) NO_SYSCA
434     288     AUE_GETSGROUPS  ALL     { int getsgroups(user_addr_t setlen, user_addr_t guidset)
435     289     AUE_SETWGROUPS  ALL     { int setwgroups(int setlen, user_addr_t guidset) NO_SYSCA
436     290     AUE_GETWGROUPS  ALL     { int getwgroups(user_addr_t setlen, user_addr_t guidset)
437     291     AUE_MKFIFO_EXTENDED     ALL     { int mkfifo_extended(user_addr_t path, uid_t uid,
438     292     AUE_MKDIR_EXTENDED      ALL     { int mkdir_extended(user_addr_t path, uid_t uid,
439     #if CONFIG_EXT_RESOLVER
440     293     AUE_IDENTITYSVC ALL     { int identitysvc(int opcode, user_addr_t message) NO_SYSC
441     #else
442     293     AUE_NULL        ALL     { int nosys(void); }
443     #endif
444     294     AUE_NULL        ALL     { int shared_region_check_np(uint64_t *start_address) NO_S
445     295     AUE_NULL        ALL     { int nosys(void); } { old shared_region_map_np }
446     296     AUE_NULL        ALL     { int vm_pressure_monitor(int wait_for_pressure, int nsecs
447     #if PSYNCH
448     297     AUE_NULL        ALL     { uint32_t psynch_rw_longrdlock(user_addr_t rwlock, uint32
449     298     AUE_NULL        ALL     { uint32_t psynch_rw_yieldwrlock(user_addr_t rwlock, uint3
450     299     AUE_NULL        ALL     { int psynch_rw_downgrade(user_addr_t rwlock, uint32_t lge
451     300     AUE_NULL        ALL     { uint32_t psynch_rw_upgrade(user_addr_t rwlock, uint32_t
```

```
452     301     AUE_NULL        ALL     { uint32_t psynch_mutexwait(user_addr_t mutex,  uint32_t m
453     302     AUE_NULL        ALL     { uint32_t psynch_mutexdrop(user_addr_t mutex,  uint32_t m
454     303     AUE_NULL        ALL     { uint32_t psynch_cvbroad(user_addr_t cv, uint64_t cvlsgen
455     304     AUE_NULL        ALL     { uint32_t psynch_cvsignal(user_addr_t cv, uint64_t cvlsge
456     305     AUE_NULL        ALL     { uint32_t psynch_cvwait(user_addr_t cv, uint64_t cvlsgen,
457     306     AUE_NULL        ALL     { uint32_t psynch_rw_rdlock(user_addr_t rwlock, uint32_t l
458     307     AUE_NULL        ALL     { uint32_t psynch_rw_wrlock(user_addr_t rwlock, uint32_t l
459     308     AUE_NULL        ALL     { uint32_t psynch_rw_unlock(user_addr_t rwlock, uint32_t l
460     309     AUE_NULL        ALL     { uint32_t psynch_rw_unlock2(user_addr_t rwlock, uint32_t
461     #else
462     297     AUE_NULL        ALL     { int nosys(void); } { old reset_shared_file }
463     298     AUE_NULL        ALL     { int nosys(void); } { old new_system_shared_regions }
464     299     AUE_NULL        ALL     { int enosys(void); } { old shared_region_map_file_np }
465     300     AUE_NULL        ALL     { int enosys(void); } { old shared_region_make_private_np
466     301     AUE_NULL        ALL     { int nosys(void); }
467     302     AUE_NULL        ALL     { int nosys(void); }
468     303     AUE_NULL        ALL     { int nosys(void); }
469     304     AUE_NULL        ALL     { int nosys(void); }
470     305     AUE_NULL        ALL     { int nosys(void); }
471     306     AUE_NULL        ALL     { int nosys(void); }
472     307     AUE_NULL        ALL     { int nosys(void); }
473     308     AUE_NULL        ALL     { int nosys(void); }
474     309     AUE_NULL        ALL     { int nosys(void); }
475     #endif
476     310     AUE_GETSID      ALL     { int getsid(pid_t pid); }
477     311     AUE_SETTIDWITHPID       ALL     { int settid_with_pid(pid_t pid, int assume) NO_SY
478     #if PSYNCH
479     312     AUE_NULL        ALL     { int psynch_cvclrprepost(user_addr_t cv, uint32_t cvgen,
480     #else
481     312     AUE_NULL        ALL     { int nosys(void); } { old __pthread_cond_timedwait }
482     #endif
483     313     AUE_NULL        ALL     { int aio_fsync(int op, user_addr_t aiocbp); }
484     314     AUE_NULL        ALL     { user_ssize_t aio_return(user_addr_t aiocbp); }
485     315     AUE_NULL        ALL     { int aio_suspend(user_addr_t aiocblist, int nent, user_ad
486     316     AUE_NULL        ALL     { int aio_cancel(int fd, user_addr_t aiocbp); }
487     317     AUE_NULL        ALL     { int aio_error(user_addr_t aiocbp); }
488     318     AUE_NULL        ALL     { int aio_read(user_addr_t aiocbp); }
489     319     AUE_NULL        ALL     { int aio_write(user_addr_t aiocbp); }
490     320     AUE_LIOLISTIO   ALL     { int lio_listio(int mode, user_addr_t aiocblist, int nent
491     321     AUE_NULL        ALL     { int nosys(void); } { old __pthread_cond_wait }
492     322     AUE_IOPOLICYSYS ALL     { int iopolicysys(int cmd, void *arg) NO_SYSCALL_STUB; }
493     323     AUE_NULL        ALL     { int process_policy(int scope, int action, int policy, in
494     324     AUE_MLOCKALL    ALL     { int mlockall(int how); }
495     325     AUE_MUNLOCKALL  ALL     { int munlockall(int how); }
496     326     AUE_NULL        ALL     { int nosys(void); }
497     327     AUE_ISSETUGID   ALL     { int issetugid(void); }
498     328     AUE_PTHREADKILL ALL     { int __pthread_kill(int thread_port, int sig); }
499     329     AUE_PTHREADSIGMASK      ALL     { int __pthread_sigmask(int how, user_addr_t set,
500     330     AUE_SIGWAIT     ALL     { int __sigwait(user_addr_t set, user_addr_t sig); }
501     331     AUE_NULL        ALL     { int __disable_threadsignal(int value); }
502     332     AUE_NULL        ALL     { int __pthread_markcancel(int thread_port); }
503     333     AUE_NULL        ALL     { int __pthread_canceled(int  action); }
504
```

```
505     ;#if OLD_SEMWAIT_SIGNAL
506     ;334    AUE_NULL        ALL     { int nosys(void); }   { old __semwait_signal }
507     ;#else
508     334     AUE_SEMWAITSIGNAL       ALL     { int __semwait_signal(int cond_sem, int mutex_sem
509     ;#endif
510
511     335     AUE_NULL        ALL     { int nosys(void); }   { old utrace }
512     336     AUE_PROCINFO    ALL     { int proc_info(int32_t callnum,int32_t pid,uint32_t flavo
513     #if SENDFILE
514     337     AUE_SENDFILE    ALL     { int sendfile(int fd, int s, off_t offset, off_t *nbytes,
515     #else /* !SENDFILE */
516     337     AUE_NULL        ALL     { int nosys(void); }
517     #endif /* SENDFILE */
518     338     AUE_STAT64      ALL     { int stat64(user_addr_t path, user_addr_t ub); }
519     339     AUE_FSTAT64     ALL     { int sys_fstat64(int fd, user_addr_t ub); }
520     340     AUE_LSTAT64     ALL     { int lstat64(user_addr_t path, user_addr_t ub); }
521     341     AUE_STAT64_EXTENDED     ALL     { int stat64_extended(user_addr_t path, user_addr_
522     342     AUE_LSTAT64_EXTENDED    ALL     { int lstat64_extended(user_addr_t path, user_addr
523     343     AUE_FSTAT64_EXTENDED    ALL     { int sys_fstat64_extended(int fd, user_addr_t ub,
524     344     AUE_GETDIRENTRIES64     ALL     { user_ssize_t getdirentries64(int fd, void *buf,
525     345     AUE_STATFS64    ALL     { int statfs64(char *path, struct statfs64 *buf); }
526     346     AUE_FSTATFS64   ALL     { int fstatfs64(int fd, struct statfs64 *buf); }
527     347     AUE_GETFSSTAT64 ALL     { int getfsstat64(user_addr_t buf, int bufsize, int flags)
528     348     AUE_NULL        ALL     { int __pthread_chdir(user_addr_t path); }
529     349     AUE_NULL        ALL     { int __pthread_fchdir(int fd); }
530     350     AUE_AUDIT       ALL     { int audit(void *record, int length); }
531     351     AUE_AUDITON     ALL     { int auditon(int cmd, void *data, int length); }
532     352     AUE_NULL        ALL     { int nosys(void); }
533     353     AUE_GETAUID     ALL     { int getauid(au_id_t *auid); }
534     354     AUE_SETAUID     ALL     { int setauid(au_id_t *auid); }
535     355     AUE_NULL        ALL     { int nosys(void); }    { old getaudit }
536     356     AUE_NULL        ALL     { int nosys(void); }    { old setaudit }
537     357     AUE_GETAUDIT_ADDR       ALL     { int getaudit_addr(struct auditinfo_addr *auditin
538     358     AUE_SETAUDIT_ADDR       ALL     { int setaudit_addr(struct auditinfo_addr *auditin
539     359     AUE_AUDITCTL    ALL     { int auditctl(char *path); }
540     #if CONFIG_WORKQUEUE
541     360     AUE_NULL        ALL     { user_addr_t bsdthread_create(user_addr_t func, user_addr
542     361     AUE_NULL        ALL     { int bsdthread_terminate(user_addr_t stackaddr, size_t fr
543     #else
544     360     AUE_NULL        ALL     { int nosys(void); }
545     361     AUE_NULL        ALL     { int nosys(void); }
546     #endif /* CONFIG_WORKQUEUE */
547     362     AUE_KQUEUE      ALL     { int kqueue(void); }
548     363     AUE_NULL        ALL     { int kevent(int fd, const struct kevent *changelist, int
549     364     AUE_LCHOWN      ALL     { int lchown(user_addr_t path, uid_t owner, gid_t group) N
550     365     AUE_NULL        ALL     { int nosys(void); }    { old stack_snapshot }
551     #if CONFIG_WORKQUEUE
552     366     AUE_NULL        ALL     { int bsdthread_register(user_addr_t threadstart, user_add
553     367     AUE_WORKQOPEN   ALL     { int workq_open(void) NO_SYSCALL_STUB; }
554     368     AUE_WORKQOPS    ALL     { int workq_kernreturn(int options, user_addr_t item, int
555     #else
556     366     AUE_NULL        ALL     { int nosys(void); }
```

```
557     367     AUE_NULL        ALL     { int nosys(void); }
558     368     AUE_NULL        ALL     { int nosys(void); }
559     #endif /* CONFIG_WORKQUEUE */
560     369     AUE_NULL        ALL     { int kevent64(int fd, const struct kevent64_s *changelist
561     #if OLD_SEMWAIT_SIGNAL
562     370     AUE_SEMWAITSIGNAL       ALL     { int __old_semwait_signal(int cond_sem, int mutex
563     371     AUE_SEMWAITSIGNAL       ALL     { int __old_semwait_signal_nocancel(int cond_sem,
564     #else
565     370     AUE_NULL        ALL     { int nosys(void); }   { old __semwait_signal }
566     371     AUE_NULL        ALL     { int nosys(void); }   { old __semwait_signal }
567     #endif
568     372     AUE_NULL        ALL     { uint64_t thread_selfid (void) NO_SYSCALL_STUB; }
569     373     AUE_LEDGER      ALL     { int ledger(int cmd, caddr_t arg1, caddr_t arg2, caddr_t
570     374     AUE_NULL        ALL     { int kevent_qos(int fd, const struct kevent_qos_s *change
571     375     AUE_NULL        ALL     { int kevent_id(uint64_t id, const struct kevent_qos_s *ch
572     376     AUE_NULL        ALL     { int nosys(void); }
573     377     AUE_NULL        ALL     { int nosys(void); }
574     378     AUE_NULL        ALL     { int nosys(void); }
575     379     AUE_NULL        ALL     { int nosys(void); }
576     380     AUE_MAC_EXECVE  ALL     { int __mac_execve(char *fname, char **argp, char **envp,
577     #if CONFIG_MACF
578     381     AUE_MAC_SYSCALL ALL     { int __mac_syscall(char *policy, int call, user_addr_t ar
579     382     AUE_MAC_GET_FILE        ALL     { int __mac_get_file(char *path_p, struct mac *mac
580     383     AUE_MAC_SET_FILE        ALL     { int __mac_set_file(char *path_p, struct mac *mac
581     384     AUE_MAC_GET_LINK        ALL     { int __mac_get_link(char *path_p, struct mac *mac
582     385     AUE_MAC_SET_LINK        ALL     { int __mac_set_link(char *path_p, struct mac *mac
583     386     AUE_MAC_GET_PROC        ALL     { int __mac_get_proc(struct mac *mac_p); }
584     387     AUE_MAC_SET_PROC        ALL     { int __mac_set_proc(struct mac *mac_p); }
585     388     AUE_MAC_GET_FD  ALL     { int __mac_get_fd(int fd, struct mac *mac_p); }
586     389     AUE_MAC_SET_FD  ALL     { int __mac_set_fd(int fd, struct mac *mac_p); }
587     390     AUE_MAC_GET_PID ALL     { int __mac_get_pid(pid_t pid, struct mac *mac_p); }
588     #else
589     381     AUE_MAC_SYSCALL ALL     { int enosys(void); }
590     382     AUE_MAC_GET_FILE        ALL     { int nosys(void); }
591     383     AUE_MAC_SET_FILE        ALL     { int nosys(void); }
592     384     AUE_MAC_GET_LINK        ALL     { int nosys(void); }
593     385     AUE_MAC_SET_LINK        ALL     { int nosys(void); }
594     386     AUE_MAC_GET_PROC        ALL     { int nosys(void); }
595     387     AUE_MAC_SET_PROC        ALL     { int nosys(void); }
596     388     AUE_MAC_GET_FD  ALL     { int nosys(void); }
597     389     AUE_MAC_SET_FD  ALL     { int nosys(void); }
598     390     AUE_MAC_GET_PID ALL     { int nosys(void); }
599     #endif
600     391     AUE_NULL        ALL     { int enosys(void); }
601     392     AUE_NULL        ALL     { int enosys(void); }
602     393     AUE_NULL        ALL     { int enosys(void); }
603     394     AUE_SELECT      ALL     { int pselect(int nd, u_int32_t *in, u_int32_t *ou, u_int3
604     395     AUE_SELECT      ALL     { int pselect_nocancel(int nd, u_int32_t *in, u_int32_t *o
605     396     AUE_NULL        ALL     { user_ssize_t read_nocancel(int fd, user_addr_t cbuf, use
606     397     AUE_NULL        ALL     { user_ssize_t write_nocancel(int fd, user_addr_t cbuf, us
607     398     AUE_OPEN_RWTC   ALL     { int open_nocancel(user_addr_t path, int flags, int mode)
608     399     AUE_CLOSE       ALL     { int sys_close_nocancel(int fd) NO_SYSCALL_STUB; }
609     400     AUE_WAIT4       ALL     { int wait4_nocancel(int pid, user_addr_t status, int opti
```

```
610     #if SOCKETS
611     401     AUE_RECVMSG     ALL     { int recvmsg_nocancel(int s, struct msghdr *msg, int flag
612     402     AUE_SENDMSG     ALL     { int sendmsg_nocancel(int s, caddr_t msg, int flags) NO_S
613     403     AUE_RECVFROM    ALL     { int recvfrom_nocancel(int s, void *buf, size_t len, int
614     404     AUE_ACCEPT      ALL     { int accept_nocancel(int s, caddr_t name, socklen_t    *a
615     #else
616     401     AUE_NULL        ALL     { int nosys(void); }
617     402     AUE_NULL        ALL     { int nosys(void); }
618     403     AUE_NULL        ALL     { int nosys(void); }
619     404     AUE_NULL        ALL     { int nosys(void); }
620     #endif /* SOCKETS */
621     405     AUE_MSYNC       ALL     { int msync_nocancel(caddr_t addr, size_t len, int flags)
622     406     AUE_FCNTL       ALL     { int sys_fcntl_nocancel(int fd, int cmd, long arg) NO_SYS
623     407     AUE_SELECT      ALL     { int select_nocancel(int nd, u_int32_t *in, u_int32_t *ou
624     408     AUE_FSYNC       ALL     { int fsync_nocancel(int fd) NO_SYSCALL_STUB; }
625     #if SOCKETS
626     409     AUE_CONNECT     ALL     { int connect_nocancel(int s, caddr_t name, socklen_t name
627     #else
628     409     AUE_NULL        ALL     { int nosys(void); }
629     #endif /* SOCKETS */
630     410     AUE_NULL        ALL     { int sigsuspend_nocancel(sigset_t mask) NO_SYSCALL_STUB;
631     411     AUE_READV       ALL     { user_ssize_t readv_nocancel(int fd, struct iovec *iovp,
632     412     AUE_WRITEV      ALL     { user_ssize_t writev_nocancel(int fd, struct iovec *iovp,
633     #if SOCKETS
634     413     AUE_SENDTO      ALL     { int sendto_nocancel(int s, caddr_t buf, size_t len, int
635     #else
636     413     AUE_NULL        ALL     { int nosys(void); }
637     #endif /* SOCKETS */
638     414     AUE_PREAD       ALL     { user_ssize_t pread_nocancel(int fd, user_addr_t buf, use
639     415     AUE_PWRITE      ALL     { user_ssize_t pwrite_nocancel(int fd, user_addr_t buf, us
640     416     AUE_WAITID      ALL     { int waitid_nocancel(idtype_t idtype, id_t id, siginfo_t
641     417     AUE_POLL        ALL     { int poll_nocancel(struct pollfd *fds, u_int nfds, int ti
642     #if SYSV_MSG
643     418     AUE_MSGSND      ALL     { int msgsnd_nocancel(int msqid, void *msgp, size_t msgsz,
644     419     AUE_MSGRCV      ALL     { user_ssize_t msgrcv_nocancel(int msqid, void *msgp, size
645     #else
646     418     AUE_NULL        ALL     { int nosys(void); }
647     419     AUE_NULL        ALL     { int nosys(void); }
648     #endif
649     420     AUE_SEMWAIT     ALL     { int sem_wait_nocancel(sem_t *sem) NO_SYSCALL_STUB; }
650     421     AUE_NULL        ALL     { int aio_suspend_nocancel(user_addr_t aiocblist, int nent
651     422     AUE_SIGWAIT     ALL     { int __sigwait_nocancel(user_addr_t set, user_addr_t sig)
652     ;#if OLD_SEMWAIT_SIGNAL
653     ;423    AUE_NULL        ALL     { int nosys(void); }   { old __semwait_signal_nocancel }
654     ;#else
655     423     AUE_SEMWAITSIGNAL       ALL     { int __semwait_signal_nocancel(int cond_sem, int
656     ;#endif
657     424     AUE_MAC_MOUNT   ALL     { int __mac_mount(char *type, char *path, int flags, caddr
658     #if CONFIG_MACF
659     425     AUE_MAC_GET_MOUNT       ALL     { int __mac_get_mount(char *path, struct mac *mac_
660     #else
661     425     AUE_MAC_GET_MOUNT       ALL     { int nosys(void); }
```

```
662     #endif
663     426     AUE_MAC_GETFSSTAT       ALL     { int __mac_getfsstat(user_addr_t buf, int bufsize
664     427     AUE_FSGETPATH   ALL     { user_ssize_t fsgetpath(user_addr_t buf, size_t bufsize,
665     428     AUE_NULL        ALL     { mach_port_name_t audit_session_self(void); }
666     429     AUE_NULL        ALL     { int audit_session_join(mach_port_name_t port); }
667     430     AUE_NULL        ALL     { int sys_fileport_makeport(int fd, user_addr_t portnamep)
668     431     AUE_NULL        ALL     { int sys_fileport_makefd(mach_port_name_t port); }
669     432     AUE_NULL        ALL     { int audit_session_port(au_asid_t asid, user_addr_t portn
670     433     AUE_NULL        ALL     { int pid_suspend(int pid); }
671     434     AUE_NULL        ALL     { int pid_resume(int pid); }
672     #if CONFIG_EMBEDDED
673     435     AUE_NULL        ALL     { int pid_hibernate(int pid); }
674     #else
675     435     AUE_NULL        ALL     { int nosys(void); }
676     #endif
677     #if SOCKETS
678     436     AUE_NULL        ALL     { int pid_shutdown_sockets(int pid, int level); }
679     #else
680     436     AUE_NULL        ALL     { int nosys(void); }
681     #endif
682     437     AUE_NULL        ALL     { int nosys(void); } { old shared_region_slide_np }
683     438     AUE_NULL        ALL     { int shared_region_map_and_slide_np(int fd, uint32_t coun
684     439     AUE_NULL        ALL     { int kas_info(int selector, void *value, size_t *size); }
685     #if CONFIG_MEMORYSTATUS
686     440     AUE_NULL        ALL     { int memorystatus_control(uint32_t command, int32_t pid,
687     #else
688     440     AUE_NULL        ALL     { int nosys(void); }
689     #endif
690     441     AUE_OPEN_RWTC   ALL     { int guarded_open_np(user_addr_t path, const guardid_t *g
691     442     AUE_CLOSE       ALL     { int guarded_close_np(int fd, const guardid_t *guard); }
692     443     AUE_KQUEUE      ALL     { int guarded_kqueue_np(const guardid_t *guard, u_int guar
693     444     AUE_NULL        ALL     { int change_fdguard_np(int fd, const guardid_t *guard, u_
694     445     AUE_USRCTL      ALL     { int usrctl(uint32_t flags); }
695     446     AUE_NULL        ALL     { int proc_rlimit_control(pid_t pid, int flavor, void *arg
696     #if SOCKETS
697     447     AUE_CONNECT     ALL     { int connectx(int socket, const sa_endpoints_t *endpoints
698     448     AUE_NULL        ALL     { int disconnectx(int s, sae_associd_t aid, sae_connid_t c
699     449     AUE_NULL        ALL     { int peeloff(int s, sae_associd_t aid); }
700     450     AUE_SOCKET      ALL     { int socket_delegate(int domain, int type, int protocol,
701     #else
702     447     AUE_NULL        ALL     { int nosys(void); }
703     448     AUE_NULL        ALL     { int nosys(void); }
704     449     AUE_NULL        ALL     { int nosys(void); }
705     450     AUE_NULL        ALL     { int nosys(void); }
706     #endif /* SOCKETS */
707     451     AUE_NULL        ALL     { int telemetry(uint64_t cmd, uint64_t deadline, uint64_t
708     #if CONFIG_PROC_UUID_POLICY
709     452     AUE_NULL        ALL     { int proc_uuid_policy(uint32_t operation, uuid_t uuid, si
710     #else
711     452     AUE_NULL        ALL     { int nosys(void); }
712     #endif
713     #if CONFIG_MEMORYSTATUS
714     453     AUE_NULL        ALL     { int memorystatus_get_level(user_addr_t level); }
```

```
714     453     AUE_NULL        ALL     { int memorystatus_get_level(user_addr_t level); }
715     #else
716     453     AUE_NULL        ALL     { int nosys(void); }
717     #endif
718     454     AUE_NULL        ALL     { int system_override(uint64_t timeout, uint64_t flags); }
719     455     AUE_NULL        ALL     { int vfs_purge(void); }
720     456     AUE_NULL        ALL     { int sfi_ctl(uint32_t operation, uint32_t sfi_class, uint
721     457     AUE_NULL        ALL     { int sfi_pidctl(uint32_t operation, pid_t pid, uint32_t s
722     #if CONFIG_COALITIONS
723     458     AUE_NULL        ALL     { int coalition(uint32_t operation, uint64_t *cid, uint32_
724     459     AUE_NULL        ALL     { int coalition_info(uint32_t flavor, uint64_t *cid, void
725     #else
726     458     AUE_NULL        ALL     { int enosys(void); }
727     459     AUE_NULL        ALL     { int enosys(void); }
728     #endif /* COALITIONS */
729     #if NECP
730     460 AUE_NECP    ALL { int necp_match_policy(uint8_t *parameters, size_t parameters_size, s
731     #else
732     460     AUE_NULL        ALL     { int nosys(void); }
733     #endif /* NECP */
734     461     AUE_GETATTRLISTBULK     ALL     { int getattrlistbulk(int dirfd, struct attrlist *
735     462     AUE_CLONEFILEAT ALL     { int clonefileat(int src_dirfd, user_addr_t src, int dst_
736     463     AUE_OPENAT_RWTC ALL     { int openat(int fd, user_addr_t path, int flags, int mode
737     464     AUE_OPENAT_RWTC ALL     { int openat_nocancel(int fd, user_addr_t path, int flags,
738     465     AUE_RENAMEAT    ALL     { int renameat(int fromfd, char *from, int tofd, char *to)
739     466     AUE_FACCESSAT   ALL     { int faccessat(int fd, user_addr_t path, int amode, int f
740     467     AUE_FCHMODAT    ALL     { int fchmodat(int fd, user_addr_t path, int mode, int fla
741     468     AUE_FCHOWNAT    ALL     { int fchownat(int fd, user_addr_t path, uid_t uid,gid_t g
742     469     AUE_FSTATAT     ALL     { int fstatat(int fd, user_addr_t path, user_addr_t ub, in
743     470     AUE_FSTATAT     ALL     { int fstatat64(int fd, user_addr_t path, user_addr_t ub,
744     471     AUE_LINKAT      ALL     { int linkat(int fd1, user_addr_t path, int fd2, user_addr
745     472     AUE_UNLINKAT    ALL     { int unlinkat(int fd, user_addr_t path, int flag) NO_SYSC
746     473     AUE_READLINKAT  ALL     { int readlinkat(int fd, user_addr_t path, user_addr_t buf
747     474     AUE_SYMLINKAT   ALL     { int symlinkat(user_addr_t *path1, int fd, user_addr_t pa
748     475     AUE_MKDIRAT     ALL     { int mkdirat(int fd, user_addr_t path, int mode); }
749     476     AUE_GETATTRLISTAT       ALL     { int getattrlistat(int fd, const char *path, stru
750     477     AUE_NULL        ALL     { int proc_trace_log(pid_t pid, uint64_t uniqueid); }
751     478     AUE_NULL        ALL     { int bsdthread_ctl(user_addr_t cmd, user_addr_t arg1, use
752     479     AUE_OPENBYID_RWT        ALL     { int openbyid_np(user_addr_t fsid, user_addr_t ob
753     #if SOCKETS
754     480     AUE_NULL        ALL     { user_ssize_t recvmsg_x(int s, struct msghdr_x *msgp, u_i
755     481     AUE_NULL        ALL     { user_ssize_t sendmsg_x(int s, struct msghdr_x *msgp, u_i
756     #else
757     480     AUE_NULL        ALL     { int nosys(void); }
758     481     AUE_NULL        ALL     { int nosys(void); }
759     #endif /* SOCKETS */
760     482     AUE_NULL        ALL     { uint64_t thread_selfusage(void) NO_SYSCALL_STUB; }
761     #if CONFIG_CSR
762     483     AUE_NULL        ALL     { int csrctl(uint32_t op, user_addr_t useraddr, user_addr_
763     #else
764     483     AUE_NULL        ALL     { int enosys(void); }
765     #endif /* CSR */
766     484     AUE_NULL        ALL     { int guarded_open_dprotected_np(user_addr_t path, const g
```

```
767     485     AUE_NULL        ALL     { user_ssize_t guarded_write_np(int fd, const guardid_t *g
768     486     AUE_PWRITE      ALL     { user_ssize_t guarded_pwrite_np(int fd, const guardid_t *
769     487     AUE_WRITEV      ALL     { user_ssize_t guarded_writev_np(int fd, const guardid_t *
770     488     AUE_RENAMEAT    ALL     { int renameatx_np(int fromfd, char *from, int tofd, char
771     #if CONFIG_CODE_DECRYPTION
772     489     AUE_MPROTECT    ALL     { int mremap_encrypted(caddr_t addr, size_t len, uint32_t
773     #else
774     489     AUE_NULL        ALL     { int enosys(void); }
775     #endif
776     #if NETWORKING
777     490     AUE_NETAGENT    ALL     { int netagent_trigger(uuid_t agent_uuid, size_t agent_uui
778     #else
779     490     AUE_NULL        ALL     { int nosys(void); }
780     #endif /* NETWORKING */
781     491     AUE_STACKSNAPSHOT ALL   { int stack_snapshot_with_config(int stackshot_config_vers
782     #if CONFIG_TELEMETRY
783     492     AUE_STACKSNAPSHOT ALL   { int microstackshot(user_addr_t tracebuf, uint32_t traceb
784     #else
785     492     AUE_NULL        ALL { int enosys(void); }
786     #endif /* CONFIG_TELEMETRY */
787     #if PGO
788     493     AUE_NULL        ALL     { user_ssize_t grab_pgo_data (user_addr_t uuid, int flags,
789     #else
790     493     AUE_NULL        ALL { int enosys(void); }
791     #endif
792     #if CONFIG_PERSONAS
793     494     AUE_PERSONA     ALL     { int persona(uint32_t operation, uint32_t flags, struct k
794     #else
795     494     AUE_NULL        ALL     { int enosys(void); }
796     #endif
797     495     AUE_NULL        ALL     { int enosys(void); }
798     496     AUE_NULL        ALL     { uint64_t mach_eventlink_signal(mach_port_name_t eventlin
799     497     AUE_NULL        ALL     { uint64_t mach_eventlink_wait_until(mach_port_name_t even
800     498     AUE_NULL        ALL     { uint64_t mach_eventlink_signal_wait_until(mach_port_name
801     499     AUE_NULL        ALL     { int work_interval_ctl(uint32_t operation, uint64_t work_
802     500     AUE_NULL        ALL     { int getentropy(void *buffer, size_t size); }
803     #if NECP
804     501     AUE_NECP        ALL     { int necp_open(int flags); } }
805     502     AUE_NECP        ALL     { int necp_client_action(int necp_fd, uint32_t action, uui
806     #else
807     501     AUE_NULL        ALL     { int enosys(void); }
808     502     AUE_NULL        ALL     { int enosys(void); }
809     #endif /* NECP */
810     503     AUE_NULL        ALL     { int enosys(void); }
811     504     AUE_NULL        ALL     { int enosys(void); }
812     505     AUE_NULL        ALL     { int enosys(void); }
813     506     AUE_NULL        ALL     { int enosys(void); }
814     507     AUE_NULL        ALL     { int enosys(void); }
815     508     AUE_NULL        ALL     { int enosys(void); }
816     509     AUE_NULL        ALL     { int enosys(void); }
817     510     AUE_NULL        ALL     { int enosys(void); }
818     511     AUE_NULL        ALL     { int enosys(void); }
```

```
819     512     AUE_NULL        ALL     { int enosys(void); }
820     513     AUE_NULL        ALL     { int enosys(void); }
821     514     AUE_NULL        ALL     { int enosys(void); }
822     515     AUE_NULL        ALL     { int ulock_wait(uint32_t operation, void *addr, uint64_t
823     516     AUE_NULL        ALL     { int ulock_wake(uint32_t operation, void *addr, uint64_t
824     517     AUE_FCLONEFILEAT        ALL     { int fclonefileat(int src_fd, int dst_dirfd, user
825     518     AUE_NULL        ALL     { int fs_snapshot(uint32_t op, int dirfd, user_addr_t name
826     519     AUE_NULL        ALL     { int enosys(void); }
827     520     AUE_KILL        ALL     { int terminate_with_payload(int pid, uint32_t reason_name
828     521     AUE_EXIT        ALL     { void abort_with_payload(uint32_t reason_namespace, uint6
829     #if NECP
830     522     AUE_NECP        ALL     { int necp_session_open(int flags); } }
831     523     AUE_NECP        ALL     { int necp_session_action(int necp_fd, uint32_t action, ui
832     #else /* NECP */
833     522     AUE_NULL        ALL     { int enosys(void); }
834     523     AUE_NULL        ALL     { int enosys(void); }
835     #endif /* NECP */
836     524     AUE_SETATTRLISTAT       ALL     { int setattrlistat(int fd, const char *path, stru
837     525     AUE_NET         ALL     { int net_qos_guideline(struct net_qos_param *param, uint3
838     526     AUE_FMOUNT      ALL     { int fmount(const char *type, int fd, int flags, void *da
839     527     AUE_NULL        ALL     { int ntp_adjtime(struct timex *tp); }
840     528     AUE_NULL        ALL     { int ntp_gettime(struct ntptimeval *ntvp); }
841     529     AUE_NULL        ALL     { int os_fault_with_payload(uint32_t reason_namespace, uin
842     #if CONFIG_WORKQUEUE
843     530     AUE_WORKLOOPCTL ALL     { int kqueue_workloop_ctl(user_addr_t cmd, uint64_t option
844     #else
845     530     AUE_NULL        ALL     { int enosys(void); }
846     #endif // CONFIG_WORKQUEUE
847     531     AUE_NULL        ALL     { uint64_t __mach_bridge_remote_time(uint64_t local_timest
848     #if CONFIG_COALITIONS
849     532 AUE_NULL  ALL { int coalition_ledger(uint32_t operation, uint64_t *cid, void *buffer,
850     #else
851     532   AUE_NULL    ALL { int enosys(void); }
852     #endif // CONFIG_COALITIONS
853     533     AUE_NULL        ALL     { int log_data(unsigned int tag, unsigned int flags, void
854     534 AUE_NULL    ALL     { uint64_t memorystatus_available_memory(void) NO_SYSCALL_STUB; }
855     535 AUE_NULL    ALL { int enosys(void); }
856     536     AUE_NULL        ALL     { int shared_region_map_and_slide_2_np(uint32_t files_coun
857     537     AUE_NULL        ALL { int pivot_root(const char *new_rootfs_path_before, const cha
858     538 AUE_TASKINSPECTFORPID       ALL { int task_inspect_for_pid(mach_port_name_t target_tpo
859     539 AUE_TASKREADFORPID  ALL { int task_read_for_pid(mach_port_name_t target_tport, int pid
860     540     AUE_PREADV      ALL     { user_ssize_t sys_preadv(int fd, struct iovec *iovp, int
861     541     AUE_PWRITEV     ALL     { user_ssize_t sys_pwritev(int fd, struct iovec *iovp, int
862     542     AUE_PREADV      ALL     { user_ssize_t sys_preadv_nocancel(int fd, struct iovec *i
863     543     AUE_PWRITEV     ALL     { user_ssize_t sys_pwritev_nocancel(int fd, struct iovec *
864     544     AUE_NULL        ALL     { int ulock_wait2(uint32_t operation, void *addr, uint64_t
865     545     AUE_PROCINFO    ALL     { int proc_info_extended_id(int32_t callnum, int32_t pid,
```